

PGR Platform

Role-based User Manual

Detailed step-by-step edition

Every feature every role can use — with the purpose behind it, the value it adds, the exact field-by-field sequence, what happens after each click, and what can refuse the action.

v4 · 2026-08-24 · covers F1–F6 shipped in the current build

How to read this manual

Five roles. Each has its own section listing every feature that role can use.

Every feature block has:

- **Purpose** — what problem it removes.
- **Value it adds** — why the office should trust it over the old way.
- **Before you start** — the account, permission, or prior action you need in place.
- **Step-by-step** — click-by-click and field-by-field. Sub-lines starting with → describe what happens after that step.
- **What can refuse the action, and why** — the platform's guards, and how to unblock.
- **Screenshot** — captured live so you can match it to what you see.

Role overview

Role	Sees	Signature capabilities
Institution Administrator	Everything	Settings, GDPR (merge/erase), statutory sign-off, Pattern Lab approval
PGR Administrator	All students & applications	Recruitment, admissions, student lifecycle, funding, thesis, completion
Supervisor	Only their supervisees	Caseload, log meetings, progression decide, exam-board classification
Student	Only themselves	My journey portal, personal preferences (F6 quiet hours)
Executive	Reporting only	Executive dashboard, Enterprise 360, Pattern Lab read-only

Sign in — every role

0.1 · Sign in and get to your landing page

Purpose. One entry point for every persona. Access-token JWT (15 min) plus refresh-token (14 days) mean silent renewal — you almost never see this screen twice a day.

Value it adds. No shared logins, no plaintext passwords in email, no expired session mid-form. Lockout after 5 failures for 15 minutes stops brute-force attempts locally.

Before you start

- The URL of the running platform (default `http://localhost:3000`).
- Your work email (the one the invitation was sent to).
- Your password (set from the invitation reset link).

Step-by-step

1. Open the platform URL in a browser. You are redirected to **/login**.
2. In the **Email** field, type your work email exactly as invited (case-insensitive).
→ *The field validates format inline — a bad address is refused before submit.*
3. In the **Password** field, type your password (no leading/trailing spaces).
→ *The eye toggle reveals the value if you want to confirm you typed it right.*
4. Click **Sign in**.
→ *The platform exchanges your credentials for an access-token JWT (15 min) + refresh-token (14 days). Access token stays in memory; refresh token stays in `localStorage`.*
5. You are redirected to your role's landing page — Institution Admin, PGR Admin and Executive land on **/dashboard**; Student lands on **/portal** (My journey); Supervisor lands on **/supervision**.
→ *Silent refresh runs in the background; if it ever fails, a single 401-retry re-issues the access token before your call fails to you.*
6. If you forgot your password, click **Forgot password** under the form. Enter the same email; the reset link arrives by email.

What can refuse the action, and why

- **Wrong password** → "Invalid credentials" and the failure counter increments.
- **5 failures inside 15 minutes** → account is locked; you must wait 15 minutes or use forgot-password.
- **Deactivated account** → "Account is disabled"; contact an Institution Administrator to reactivate.
- **Backend down** → the login form shows a red banner; the request logs show `/health/ready` failing.



PGR Platform

Sign in to continue

Email

Password

[Forgot password?](#)

Sign in

Demo: admin@example.com / admin123

Login page. Demo shipped with admin@example.com / admin123 (Institution Admin) and elena.ford@example.com / super123 (Supervisor).

Institution Administrator

The full-access role. Owns the institution's configuration, the identity spine, GDPR operations, statutory sign-off, and the Ask PGR write policy. Also has every PGR Administrator capability — read those in Role 2 too.

1.1 · Executive dashboard — reading the morning at a glance

Purpose. The one screen leadership opens first. Six lifecycle metrics and four administrator queues in the same fold, so nothing needing attention today can hide.

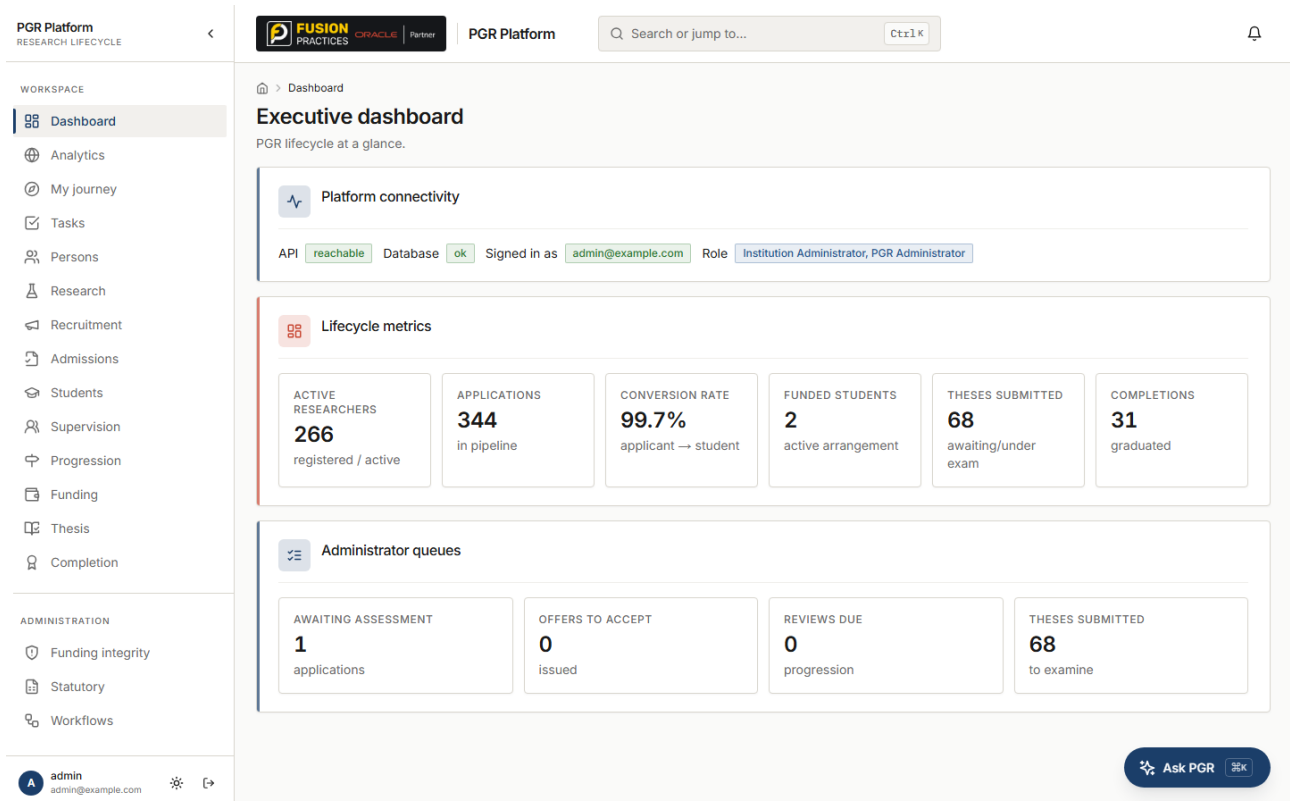
Value it adds. A single fold that answers: how many active researchers, applications in pipeline, conversion rate, funded students, theses under exam, completions. Every non-zero queue count is a specific piece of work waiting on a human decision.

Before you start

- Signed in as Institution Admin, PGR Admin or Executive.

Step-by-step

1. Land on **/dashboard** after sign-in.
→ The **Platform connectivity** strip at the top shows API reachable, DB ok, signed-in user and roles — one glance confirms the platform itself is healthy before you read the numbers.
2. Read the six **Lifecycle metrics** tiles: Active researchers · Applications · Conversion rate · Funded students · Theses submitted · Completions.
→ Numbers refresh on next page load; there is no polling here (deliberate — the dashboard is a snapshot).
3. Scan the **Administrator queues**: Awaiting assessment · Offers to accept · Reviews due · Theses submitted.
→ Each non-zero count is a link — click to jump into the specific list already filtered.
4. Use the sidebar to drill into any domain area. Or press **Ctrl/Cmd+K** to open Ask PGR from anywhere.
→ The Ask PGR launcher (bottom-right) is a fallback if the keyboard shortcut is intercepted by the OS.



Executive dashboard — 266 researchers, 344 applications, 99.7% conversion, 68 theses under exam, 31 graduated.

1.2 · Settings — Institution policy (the wired defaults)

Purpose. The institution's control panel for policy that changes runtime behaviour: supervision capacity, expected meeting interval, part-time factor, minimum funding-gap threshold, email kill-switch, correction turnarounds, assistant LLM enable flag.

Value it adds. Policy changes stop being code releases. A single PUT alters behaviour with type + range validation on the next request. Reset removes the override so the shipped default returns transparently.

Before you start

- Signed in as Institution Administrator (holds `admin.configure`).

Step-by-step

1. Open **Settings** from the sidebar.
2. Click the **Institution policy** tab.
 - You see a list of every setting the platform reads at runtime, grouped by area (supervision, lifecycle, funding, email, corrections, assistant).
3. Find the setting you want to change (for example `supervision.max_supervisees`).
 - The row shows: *current value · default value · description of what it affects*.
4. Click **Edit** in the row. Type the new value in the input.
 - Type + range validation runs client-side and again server-side. A value out of range is refused with the allowed bounds named.
5. Click **Save**.
 - The setting is committed. The **next request** that reads this setting will see the new value — no restart needed. The audit trail records who changed it and to what.
6. To revert, click **Reset to default** on the row.
 - The override is deleted; subsequent requests read the shipped default value.

What can refuse the action, and why

- **Value out of range** → refused with the exact bounds (e.g. "max_supervisees must be between 1 and 30").
- **Wrong type** → refused (e.g. passing text where an integer is required).
- **Missing admin.configure** → 403 Forbidden.

The screenshot shows the PGR Platform interface. The top navigation bar includes the PGR Platform logo, a search bar, and a notification bell. The left sidebar is divided into 'WORKSPACE' and 'ADMINISTRATION' sections. The 'Settings' page is active, showing a breadcrumb 'Settings' and a description: 'Institution configuration, reference data, user administration and your personal preferences.' Below this are four tabs: 'List of values', 'Institution policy', 'Users & roles', and 'My preferences'. The 'List of values' tab is selected, showing sub-tabs for 'Departments', 'Research areas', 'Programmes', and 'Funding sources'. A message states: 'Values still referenced by live records cannot be deleted — the platform will tell you exactly what is using them.' A table with columns 'Name', 'Code', 'In use', and 'Actions' is displayed. The first row is 'Computer Science' with code 'CS' and '399 records' in use. A '+ Add department' button is in the top right. At the bottom right is an 'Ask PGR' button. The bottom left shows the user 'admin' with email 'admin@example.com'.

Settings — the four tabs across the top: List of values, Institution policy, Users & roles, My preferences.

1.3 · Settings — List of values (departments, areas, programmes, funding sources)

Purpose. CRUD over the institution's vocabularies. Every row carries a live usage count.

Value it adds. Deletes are refused while a value is in use, with the exact reference count named ("still referenced by 2 students"). Duplicate codes are refused case-insensitively. You cannot orphan a live record by accident.

Before you start

- Signed in with `admin.configure`.

Step-by-step

1. Open **Settings** → **List of values**.
2. Click the tab for the vocabulary you want to manage: **Departments** / **Research areas** / **Programmes** / **Funding sources**.
→ *The table lists every value with its **Code**, **Name** and **In-use** count.*
3. **To add:** click + **Add ...** in the top-right.
4. Fill **Name** and **Code** (unique, case-insensitive) in the dialog. Optional fields depend on the vocabulary.
→ *Save — the row appears immediately.*
5. **To rename:** click the pencil icon on the row, edit the field, save. Existing references keep pointing at the same UUID.
6. **To delete:** click the trash icon on the row.
→ *If the value is in use, the delete is refused with the count and the specific dependent aggregate named (e.g. "still referenced by 2 students, 3 opportunities"). Fix the references first, or deactivate the value instead.*

7. **To deactivate** (softer than delete): use the **Active** toggle. Deactivated values are hidden from new pickers but still resolve on existing records.

What can refuse the action, and why

- **Duplicate code** (case-insensitive) → 409 Conflict with the existing row named.
- **Delete refused** → the response body lists what still references the value.

1.4 · Settings — Users & roles (invite, activate, assign)

Purpose. Invite, activate/deactivate and role-assign users. The invitation IS the password-reset email — no administrator ever handles a plaintext password.

Value it adds. Institution-side lockout prevents you removing your own admin access, so you cannot lock yourself out. Undo would need database surgery.

Before you start

- Institution Administrator role.

Step-by-step

1. Open **Settings** → **Users & roles**.
2. Click **Invite user** (top-right).
→ A dialog opens with fields for **Email** and multi-select **Roles**.
3. Type the invitee's email. Pick roles (Institution Administrator, PGR Administrator, Supervisor, Student, Executive) with the checkboxes.
→ The dialog warns you about assigning Institution Administrator specifically (it is the only role that carries *).
4. Click **Send invitation**.
→ The user row appears in the table with status *Invited*. An email is dispatched carrying the one-time reset URL.
5. The invitee opens the URL, sets their password, and can then sign in.
→ The token is single-use and expires; a second click on the same URL is refused.
6. **To activate / deactivate an existing user:** click the toggle in the **Active** column.
→ Deactivation takes effect on the user's next token refresh (≤ 15 min). To boot them immediately, also run `logout -all` on their user id.
7. **To reassign roles:** click the roles cell to reopen the multi-select.
→ Adding a permission via a role reaches a live session only after the token refreshes (≤ 15 min) or the user signs in again.
8. **Attempting to remove your own admin access** is refused with a specific message.
→ This lock is intentional — losing all admin permissions would require database surgery to unlock.

What can refuse the action, and why

- **Duplicate email** → the user already exists; edit that row instead of re-inviting.
- **Self-lockout** → "You cannot remove your own Institution Administrator role."

1.5 · Statutory returns — HESA profile sign-off (F1)

Purpose. A statutory return is versioned **configuration**, not code. Owners (Registry / HESA SME) attest that mappings satisfy the return's mandatory spec. Once signed off, the profile is immutable until deliberately unsigned.

Value it adds. Submitting the wrong HESA return is a governance incident. F1 refuses sign-off until every mandatory spec field is mapped AND the current cohort validates. The exact missing fields and their coding frames are named, so the fix is obvious.

Before you start

- Signed in with `reports.signoff` (defaults to Institution + PGR Administrator).
- A profile exists (create one via **+ New profile** if not — use code `HESA_STUDENT` to link to the built-in mandatory-fields registry).

Step-by-step

1. Open **Statutory** in the Administration sidebar.
2. Click the profile row (e.g. `HESA_STUDENT · 2026/27 · v1`).
→ The **Sign-off & mandatory-field readiness** panel loads.
3. Read the badge — red "Not ready — N mandatory fields unmapped" or amber "Ready to sign off" or green "Signed off".
→ The counter next to it ("`12 / 24 spec fields mapped`") gives the coverage.
4. Read the **Missing mandatory fields** table. Every row shows: field name, description, and coding frame if any.
→ The coding frame column tells you which transform to apply (e.g. field `SEXID` takes values `10, 11, 12, 13` — use transform `hesa_sex` to convert `Female / Male / Other / Unknown` to those codes).
5. For each missing field: click **+ Add field** at the bottom of the field list.
→ The *Map a field* dialog opens.
6. In the dialog: **Target field** = the missing field name (e.g. `SEXID`); **Source expression** = the dotted path over the flat student record (e.g. `person.sex`); **Transform** = the coding-frame transform (`hesa_sex`); **Allowed values** = the coding frame (paste "`10, 11, 12, 13`"); **Required** = ✓ for spec-mandatory fields.
→ Save the field. The compile panel refreshes; the missing count decreases by 1.
7. Repeat until the badge turns amber ("Ready to sign off").
→ Optionally click **Validate** — the platform runs every student against every field and reports errors before you touch generate.
8. Click **Sign off**. The **Sign off dialog** asks for optional notes.
→ Type notes such as "`Confirmed against HESA Student 2026/27 v1.2 with Registry on 2026-08-24`".
9. Click **Sign off** in the dialog.
→ If the current cohort produces any validation error, sign-off is refused with the error count; go fix and retry. If clean, the profile turns green, all mutating actions on it are refused, and the audit trail records who signed off and when.
10. To reopen for edits later, click **Unsign** in the locked banner. This is a deliberate, audited action.
→ The profile is editable again; sign-off state is cleared; run through the flow again to re-lock.

What can refuse the action, and why

- **Missing mandatory fields** → refused with the exact list, first eight named.
- **Cohort validation errors** → refused with the error count. Click **Validate** to see them per student per field.
- **Unknown transform** → refused at add-field time; the response lists every available transform.
- **Missing** `reports.signoff` → 403; the Sign-off button will not render.

PGR Platform

RESEARCH LIFECYCLE

FUSION PRACTICES

ORACLE Partner

PGR Platform

Q Search or jump to...

CtrlK

WORKSPACE

Dashboard

Analytics

My journey

Tasks

Persons

Research

Recruitment

Admissions

Students

Supervision

Progression

Funding

Thesis

Completion

ADMINISTRATION

Funding integrity

Statutory

Workflows

admin

admin@example.com

⚙️

↔️

🏠

> Statutory

Statutory returns

+ New profile

A statutory return is configuration, not code — HESA is an external specification, expressed as a versioned profile of field mappings.

📄

Report profiles

Versioned by academic year. Next year's change is an edit here, not a code release.

Code	Name	Academic year	Version	Fields	Sign-off	Status
HESA_STUDENT	HESA Student Return	2026/27	v1	12	draft	active

🛡️

Sign-off & mandatory-field readiness

A profile can be signed off only when every mandatory field in the return's published spec is mapped and the current cohort validates. Signed-off profiles are immutable until unsigned.

🚫 Not ready — 15 mandatory fields unmapped

12 / 24 spec fields mapped

Missing field	Description	Coding frame
OWNSTU	Institution's own student identifier	free text
BIRTHDATE	Date of birth (YYYYMMDD)	free text
SEXID	Sex identifier	10, 11, 12, 13
ETHNIC	Ethnicity code	10, 15, 16, 17, 18, 19, 20, 21, 22, 29, 31, 32, 33, 34, 41, 42, 43, 49, 50, 98
DISABLE	Disability indicator	00, 51, 53, 54, 55, 56, 57, 58, 96
COURSETYPE	Course type	A, B, C, D, E
MODE	Mode of study	01, 02, 03, 31
STUDYLEVEL	Level of study	D00, M11, H11, I11

Ask PGR

⌕

1.6 · Persons — merge, subject-access export, right-to-erasure (F2)

Value it adds. Duplicate records will accumulate on any real import. Merge is the only clean fix. Subject-access has a 30-day statutory clock in the UK — F2 makes it a one-click JSON download. Erasure preserves audit rows by hashing, not deleting.

- Signed in with person.gdpr (defaults to Institution Administrator only).
- For merge: both source person UUIDs known.
- For erasure: three-step confirmation ready — the action is irreversible.

1. Merge two persons
2. Open **Persons**. Click **Merge persons** in the page header (only visible if you hold person . gdpr).
→ *The Merge dialog opens with three fields.*
3. Paste the **Surviving person id** (the one you want to keep).
→ *This person's row survives, with all foreign keys rewritten onto it.*
4. Paste the **Losing person id** (the one to delete).
→ *This row will be removed at the end of the merge; its personal fields are captured as a snapshot in the merge record.*
5. Type the **Reason** ("same person — two applicant emails from Route B intake").
→ *Recorded on the immutable merge record for later review.*
6. Click **Merge**.

→ In a single transaction: every FK to the losing person is rewritten; the losing row is deleted; a person_merge_record row is written with the tables/columns touched. The toast reports "Merged — N row(s) rewritten".

7. Subject-access export

8. Open the person you want to export. Scroll to the **GDPR** card.

→ The card renders because you hold person.gdpr.

9. Click **Download subject-access export (JSON)**.

→ The platform walks every table with a person_id FK (metadata-driven — new modules auto-included). A JSON file person-{id}-subject-access.json is streamed to your browser.

10. Verify the file contents open in any JSON viewer.

→ The top-level shape: {person, related: {student:[...], funding_arrangement:[...], milestone:[...], ...}}. Hand this file to the subject.

11. Right-to-erasure

12. On the person's GDPR card, click **Erase this person**.

→ The Erase dialog opens with a red warning.

13. Type **ERASE** in capital letters in the confirmation field.

→ The Erase button un-disables only once you have typed ERASE exactly.

14. Click **Erase permanently**.

→ The person's given_name/family_name become erased; email is replaced with a one-way hash; DOB and nationality are cleared; contacts are deleted; pseudonymised_at is stamped. Audit rows and financial audit stay — hashed, not deleted.

What can refuse the action, and why

- **Merging a person into themselves** → 422 with "cannot merge a person into themselves".
- **Unknown person id** → 404 on either surviving or losing side.
- **Merging an erased person** → 409 with "cannot merge a GDPR-erased person" (either side).
- **Re-erasing an erased person** → 409; the person's pseudonymised_at is already set.

The screenshot displays the PGR Platform interface for a person named Amara Abara. The interface is divided into a sidebar on the left and a main content area on the right. The sidebar contains navigation links for Workspace (Dashboard, Analytics, My journey, Tasks, Persons) and Administration (Funding integrity, Statutory, Workflows). The main content area shows the person's profile, identities, contacts, GDPR card, and lifecycle timeline. The GDPR card includes a button to 'Download subject-access export (JSON)' and a button to 'Erase this person'. The lifecycle timeline shows the person's status as a student from 2025-05-03 to the current date.

F2 — Person detail with Contacts card (F2) and GDPR card (subject-access export + right-to-erasure).

1.7 · Persons — the identity spine (browse and search)

Purpose. One person_id across every identity the person has ever held. Concurrent identities are supported — a PGR who is also staff has both open at once.

Value it adds. Never lose the thread. Alumni confirmation, HR staff-status changes, and Route B applicant → student handoffs all reuse the same person_id.

Before you start

- Signed in with person.read.

Step-by-step

1. Open **Persons** from the sidebar.
2. Use the search bar to filter by name or email (case-insensitive substring).
→ *The list re-queries as you type; no pagination hitch.*
3. Click a person row.
→ *Person detail loads with Profile · Identities · Contacts · GDPR · Lifecycle timeline sections.*
4. The **Identities** card shows every relationship (applicant, student, employee, alumnus, researcher) with its **valid_from** / **valid_to** and source system.
→ *A relationship with valid_to = null is currently open. Concurrent identities appear as parallel bands.*
5. The **Lifecycle timeline** at the bottom stitches every event (applications, offers, student events, thesis submits, graduation) into one chronological view.
→ *Read top-down to see the person's life at the institution.*

PGR Platform RESEARCH LIFECYCLE

FUSION PRACTICES ORACLE Partner **PGR Platform** Search or jump to... Ctrl K

WORKSPACE

- Dashboard
- Analytics
- My journey
- Tasks
- Persons**
- Research
- Recruitment
- Admissions
- Students
- Supervision
- Progression
- Funding
- Thesis
- Completion

ADMINISTRATION

- Funding integrity
- Statutory
- Workflows

admin admin@example.com

Persons

One person across every identity over time.

Search by name or email...

Name	Email	Nationality	Current identities
Amara Abara	scale99s144@student.example.ac.uk	Indian	student · current
Chen Abara	scale99s271@student.example.ac.uk	Brazilian	student · current
Dara Abara	scale99s214@student.example.ac.uk	Chinese	student · current
Iris Abara	scale99s109@student.example.ac.uk	Kenyan	student · current
Iris Abara	scale99s33@student.example.ac.uk	Brazilian	student · current
Jonas Abara	scale99s150@student.example.ac.uk	German	student · current
Priya Abara	scale99s218@student.example.ac.uk	German	student · current
Priya Abara	scale99s200@student.example.ac.uk	—	student · current
Quinn Abara	scale99s23@student.example.ac.uk	Indian	student · current
Sami Abara	scale99s259@student.example.ac.uk	British	student · current
Tara Abara	scale99s143@student.example.ac.uk	Kenyan	student · current
Uma Abara	scale99s184@student.example.ac.uk	Irish	student · current
Viktor Abara	scale99s195@student.example.ac.uk	Chinese	student · current
Marcus Bell	marcus.bell@example.ac.uk	Irish	student · current employee · current
Ben Bennett	sup9934@uni.example.ac.uk	British	employee · current
Chen Bennett	scale99s84@student.example.ac.uk	British	student · current

Merge persons

Ask PGR

Persons list — Merge action available for admins holding person.gdpr.

1.8 · Ask PGR — natural-language layer with write intents (F6)

Purpose. Rules-first assistant for reads. F6 adds a two-step write path: propose → execute. Tier-3 blocked actions (graduation, exam outcome, payment approval, appeals) are refused at propose time with the exact rule cited.

Value it adds. No student data leaves the server for reads (LLM off by default). Writes are audited, actor-scoped, and gated by an institution-signed Tier-3 policy list.

Before you start

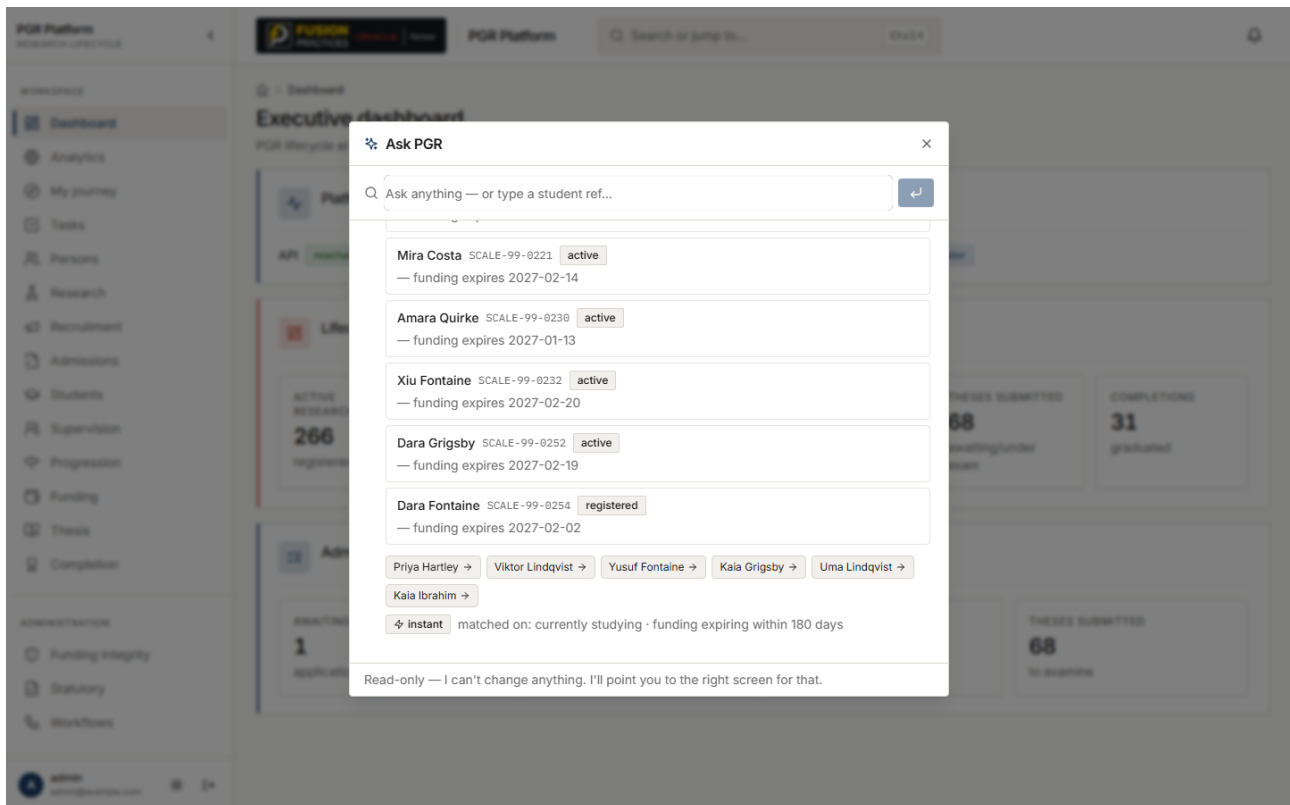
- Signed in with `assistant.use` (defaults to Institution + PGR Administrator).

Step-by-step

1. Click the floating ■ **Ask PGR** button (bottom-right of any screen) or press **Ctrl/Cmd+K**.
→ *The palette opens with an input at the top.*
2. Type your question in plain English: "who is falling through the cracks" · "whose money is running out" · "no supervision meeting in 90 days".
→ *The palette shows results as you type. Every answer carries an **Understood as** readback ("no supervision meeting in 180 days") so you can verify the interpretation.*
3. Click a suggested student to jump into their record.
→ *The palette closes; you land on the student detail.*
4. **To propose a write (F6)**
5. Ask the assistant for an action it supports (e.g. "log a 30-minute supervision meeting for student X").
→ *The palette returns an intent preview: **Action: meeting.log**, scope, and a plain-English summary of what will change.*
6. Click **Confirm** in the preview.
→ *The intent is executed inside your permission scope. The audit trail records the write with `actor_source=assistant`.*
7. If the action is on the Tier-3 blocked list (graduate, payment approval, decide milestone, decide appeal, delete document, examination outcome, correction sign-off, fee waiver approval), the propose call is refused.
→ *The response cites the exact rule: "Graduation is final and closes funding; it must be confirmed on the completion form."*
8. **To cancel a proposed intent** click Cancel in the preview.
→ *The intent moves to cancelled without side effects.*

What can refuse the action, and why

- **Tier-3 blocked action** → refused at propose time with the policy line quoted.
- **Missing** `assistant.use` → 403; the palette never opens (the launcher is hidden).
- **LLM disabled** (default) → only rules-matched queries work. Unmatched queries return concrete suggestions, never a guess.



Ask PGR answering 'funding expiring in 6 months' — matched deterministically in ≈ 1 ms.

1.9 · Pattern Lab — approving a governed ML model

Purpose. Discover → Validate → Train → Approve → Deploy → Monitor → Retrain. Governed institutional machine learning; ml . approve defaults to Institution Admin only.

Value it adds. Approver separation is enforced (promoter $\neq$ trainer). Nothing auto-promotes. Every prediction carries perturbation factors. The system caught its own first real issue (matured AUC 0.48 vs trained 0.70, drift in four features) — surfaced for review without acting.

Before you start

- Signed in with ml . approve.
- A candidate model has been trained by a different user (approver separation).

Step-by-step

1. Open **Advanced** → **Pattern Lab**.
2. Click the **Governance** tab.
 - The list of candidate versions loads, each with metrics: CV AUC, out-of-fold precision/recall, baseline delta.
3. Click a candidate row to open its **model card**.
 - The card shows: training dataset content hash, feature importances, permutation importance, computed limitations, and the trainer's user id.
4. Verify the trainer's user id is not you (approver separation).
 - The Approve button is disabled if the trainer is you.
5. Read the rationale field. Type your approval rationale ("AUC 0.72 beats baseline 0.61; drift on cohort-30d is acceptable; approved for production").
 - Rationale is mandatory — the button stays disabled until it has ≥ 20 characters.
6. Click **Approve** → **Promote to production**.
 - The candidate moves to state approved, then production. Batch scoring picks it up on the next run. The governance log records who approved with what rationale.

7. Under **Monitoring**, watch the model's PSI drift and calibration-in-the-wild.

→ If health verdict turns amber, review the reasons named; if red, initiate a retrain (nothing auto-retrains).

What can refuse the action, and why

- **Self-approval attempt** → refused, "approver must differ from trainer".
- **Empty rationale** → refused, minimum 20 characters.
- **Candidate below baseline** → the row is marked failed and cannot be approved.

The screenshot displays the PGR Platform interface, specifically the 'Pattern Lab' section. The left sidebar contains navigation links for Workspace (Dashboard, Analytics, My journey, Tasks, Persons, Research, Recruitment, Admissions, Students, Supervision, Progression, Funding, Thesis, Completion) and Administration (Funding integrity, Statutory, Workflows). The main content area is titled 'Pattern Lab' and includes a search bar and a 'New analysis' button. Below the title, there are tabs for Overview, Discover, Models, Predictions, and Monitoring. The 'Overview' tab is active, showing 'Active models' (2 models) and 'Recent discoveries'. The 'Active models' section lists two models: 'Progression Delay Risk model' (AUC 0.609) and 'Funding Continuity Risk model' (AUC 0.702), both with 'candidate' status. The 'Recent discoveries' section shows two entries, both marked as 'significant' and '1.7x the rate'. The 'Data health' section is also visible, with a 'Ask PGR' button. The bottom of the interface shows the user 'admin' and a 'Pattern Lab' title.

Pattern Lab — approver separation prevents self-promotion.

PGR Administrator

The day-to-day operational role that runs the postgraduate research office. Covers the full applicant → alumnus lifecycle, statutory returns (except sign-off), reconciliation, and audit. Everything in Role 1 that isn't sign-off, GDPR or Pattern Lab approval is here too.

2.1 · Research — awards (read-only) and demand (state machine)

Purpose. Life starts before the applicant. Awards are grant references mastered by Research Office (read-only, indicated by the padlock). Demand rows walk proposed → approved → posted; each posted demand produces an opportunity.

Value it adds. Every position, applicant and student can be traced back to the award and demand that created it. Provenance is a top-of-funnel property, not an afterthought.

Before you start

- Signed in with `recruitment.read` to browse, `recruitment.write` to move demand states.

Step-by-step

1. Open **Research** from the sidebar.
2. **Browsing awards**
3. Click the **Awards** tab.
 - Each award shows its external reference, title, funder, start/end dates, and total value. Rows carry a padlock — awards are externally mastered so you cannot edit them here.
4. Click an award row for its detail view.
 - The *Related* section lists every demand and every downstream opportunity/student created against this award — the provenance chain in one place.
5. **Creating and moving demand**
6. Click the **Demand** tab, then **+ New demand**.
 - Fill title, research area, department, whether it links to an award, target start date, expected duration.
7. Save — the demand is created in state **proposed**.
 - Proposed demand does not produce an opportunity yet.
8. Click **Approve** (with `recruitment.write`).
 - State moves to **approved**. Optional gate for institutional review.
9. Click **Post**.
 - State moves to **posted** and an opportunity is created in Recruitment ready for applications. The provenance link (`research_demand_id` + `research_award_id`) is populated on that opportunity.

What can refuse the action, and why

- **Trying to edit an award** → refused (externally mastered, source-of-record is Research Office).
- **Skipping states** (e.g. proposed → posted directly) → refused; move through the state machine step by step.

PGR Platform
<

FUSION PRACTICES ORACLE Partner

Q Search or jump to... Ctrl K

WORKSPACE

- Dashboard
- Analytics
- My journey
- Tasks
- Persons
- Research
- Recruitment
- Admissions
- Students
- Supervision
- Progression
- Funding
- Thesis
- Completion

ADMINISTRATION

- Funding integrity
- Statutory
- Workflows

Research

Where a PGR position comes from: an award funds a demand, a demand becomes a position.

[Research demand](#)
[Awards](#)
[Supervisor match](#)
[Relationship map](#)

Demand is the recorded *need* for a researcher. It becomes a recruitable position only once it is approved and positioned. + Raise demand

Title	Award	Places	Target start	Status	
PGR researcher for Robotics Programme Grant	SCALE-AWD-99-020	2	2026-11-20	positioned	Move to... ▼
PGR researcher for Climate Systems Programme Grant	SCALE-AWD-99-019	3	2026-11-20	positioned	Move to... ▼
PGR researcher for Machine Learning Programme Grant	SCALE-AWD-99-018	3	2026-11-20	positioned	Move to... ▼
PGR researcher for Health Informatics Programme Grant	SCALE-AWD-99-017	2	2026-11-20	positioned	Move to... ▼
PGR researcher for Robotics Programme Grant	SCALE-AWD-99-016	2	2026-11-20	positioned	Move to... ▼
PGR researcher for Computational Biology Programme Grant	SCALE-AWD-99-015	2	2026-11-20	positioned	Move to... ▼
PGR researcher for Machine Learning Programme Grant	SCALE-AWD-99-014	3	2026-11-20	positioned	Move to... ▼
PGR researcher for Materials Science Programme Grant	SCALE-AWD-99-013	1	2026-11-20	positioned	Move to... ▼
PGR researcher for Robotics Programme Grant	SCALE-AWD-99-012	2	2026-11-20	positioned	Move to... ▼
PGR researcher for Materials Science Programme Grant	SCALE-AWD-99-011	3	2026-11-20	positioned	Move to... ▼
PGR researcher for Materials Science Programme Grant	SCALE-AWD-99-010	3	2026-11-20	positioned	Move to... ▼
PGR researcher for Materials Science Programme Grant	SCALE-AWD-99-009	2	2026-11-20	positioned	M... ▼
PGR researcher for Climate Systems Programme Grant	SCALE-AWD-99-008	2	2026-11-20	positioned	Move to... ▼

admin
admin@example.com

⚙️
➔

Ask PGR

2.2 · Recruitment — pipeline with route badges + filter (F3)

Value it adds. Two entry routes reach Enterprise 360 and the statutory return correctly. The filter chip lets you triage "show me all Route B this week" without opening each record.

- `recruitment.read`.

1. Open **Recruitment**.
2. Under **Opportunities**, filter by department, research area or status.
→ Each row shows **places filled / available** live. A full opportunity is refused any new offer.
3. Click an opportunity, then click **Provenance**.
→ A dialog walks award → demand → opportunity → shortlist so you can see everything that made this position exist.
4. Click the **Applications** tab.
→ The pipeline stage counts appear at the top; the applications list is below.
5. In the **Route**: chip filter, click **Route A · opportunity** or **Route B · student proposal** to narrow.
→ The 'Showing X of Y' counter updates. Click **All** to clear the filter.
6. Click an application row.
→ The application detail loads with stage history, assessments, references (F3) and interviews (F3).
7. When ready, click **Recommend for offer** in the application detail.
→ This creates a draft offer in Admissions attached to the same application.

PGR Platform

RESEARCH LIFECYCLE

FUSION PRACTICES

ORACLE Partner

PGR Platform

Q Search or jump to...

Ctrl K

WORKSPACE

Dashboard

Analytics

My journey

Tasks

Persons

Research

Recruitment

Admissions

Students

Supervision

Progression

Funding

Thesis

Completion

ADMINISTRATION

Funding integrity

Statutory

Workflows

admin

admin@example.com

Recruitment

Opportunities and the application pipeline.

Opportunities

Applications

APPLICANT

1

CONVERTED

343

ROUTE:

All

Route A - opportunity

Route B - student proposal

Showing 100 of 100

Application	Route	Stage	Submitted
d6858f57...	Route A	converted	—
ae81f628...	Route A	converted	—
c8bd80c0...	Route A	converted	—
1b6401b0...	Route A	converted	—
7df71fdd...	Route A	converted	—
4a925836...	Route A	converted	—
eb1a129f...	Route A	converted	—
68d4af77...	Route A	converted	—
27daede9...	Route A	converted	—
80c07e70...	Route A	converted	—
d5a36a12...	Route A	converted	—
0356f003...	Route A	converted	—
cd062626...	Route A	converted	—

Ask PGR

F3 — Route badges + filter chips. 'Showing 100 of 100' updates as you filter.

2.3 · Recruitment — references and interviews as first-class objects (F3)

Purpose. References and interviews used to live in email + Word. F3 makes them first-class: **ReferenceRequest** with a one-time token URL for the referee (unauthenticated submit); **Interview** with panellists, schedule and outcome.

Value it adds. Every referee response and every panel decision is on the audit trail. Duplicate token submission is refused; duplicate panellist on one interview is refused.

Before you start

- `recruitment.write` to create; the referee needs only the token URL.

Step-by-step

1. **Requesting a reference**
2. Open the application. In the **References** card, click **Request reference**.
→ *The Request reference dialog opens.*
3. Fill **Referee name**, **Referee email**, and optional **Affiliation** ("University of Elsewhere").
→ *Submit.*
4. The response returns the submit URL (`/public/references/{token}`). Copy this URL and email it to the referee.
→ *The token is hashed server-side; the raw value is shown to you only once at creation time.*
5. The referee opens the URL (no login), pastes their response text (and optionally a document ref), submits.
→ *The row moves from requested to received. A second submit on the same token is refused with 409.*
6. **Scheduling an interview**
7. In the **Interviews** card, click **Schedule interview**.
→ *Dialog: scheduled datetime, location.*

8. Save. Then click the row to open detail. Click **Add panellist** and pick the person by id, role (chair / assessor).

→ Adding the same person twice on one interview is refused.

9. After the interview, click **Record outcome** and pick **proceed / hold / reject**. Add notes.

→ The interview moves to completed with the outcome recorded on the audit trail.

What can refuse the action, and why

- **Bad reference token** → 404 "Reference token not recognised".
- **Re-submitting a used token** → 409 "Reference already received".
- **Duplicate panellist** → 409 "That person is already on the panel".

The screenshot displays the 'Application' detail page in the PGR Platform. The page is divided into several sections: 'Overview' showing applicant details (Omar Okafor, route: opportunity led, stage: converted), 'Advance stage' with fields for target stage and reason, 'Record assessment' with a decision dropdown (recommended), rationale, and a 'Record' button, 'Offer' with a 'Create offer' button, and 'Stage history' showing no history. The left sidebar contains a 'Recruitment' menu. The top header includes the PGR Platform logo, Fusion Practices Oracle Partner badge, and a search bar.

Application detail — stage history, assessments, references + interviews (F3) cards.

2.4 · Admissions — offers with conditions and the visa gate (F3)

Purpose. Offers move draft → issued → accepted/declined. F3 adds **OfferCondition** (satisfy-by date, evidence document) and a **visa gate**: an application flagged `visa_required` cannot have its offer issued until the visa check is complete. Accept is refused while any condition is unsatisfied.

Value it adds. Conditional offers stop living in Word docs. UKVI compliance risk is caught at the platform edge. Accept is the single most consequential transaction — it creates the student atomically with the same `person_id`.

Before you start

- `recruitment.write`.

Step-by-step

1. Visa check

2. On the application detail, open the **Fee & visa** card. Set **Fee status** (home / overseas / channel_islands / unknown) and toggle **Visa required**.

→ If *Visa required* is on, the offer cannot issue without a completed check.

3. When the check is complete (documents seen, right-to-study confirmed), click **Complete visa check**.
→ *visa_check_completed_at is stamped with the timestamp; issue is now allowed.*
4. **Draft the offer**
5. From the application, click **Recommend for offer**. A draft offer appears in Admissions.
→ *The offer is in state **draft**. Nothing has been issued to the applicant yet.*
6. Open the offer. In the **Conditions** card click **+ Add condition**.
→ *Fill description ("Pass Masters at 2:1+") and satisfy-by date.*
7. Repeat for every condition. Each has state pending.
→ *Conditions are additive; the applicant sees them in the portal once you issue.*
8. **Issue the offer**
9. Click **Issue**.
→ *The visa gate runs — if visa_required and check is not complete, refused with "complete the visa check before issuing". Otherwise state → **issued**, timestamps recorded, notification queued to the applicant.*
10. **Accept the offer**
11. Once the applicant responds, open the offer and mark each condition **Satisfied** (with optional evidence document) or **Waived**.
→ *Both actions are audited; the row shows who satisfied/waived and when.*
12. Click **Accept**. Pick programme, study mode, start date, current user for the audit.
→ *If any condition is still pending, refused with the count and the first condition named. Fix and retry.*
13. Accept succeeds → student is created atomically: person_id reused, project linked to award if any, funding lineage populated, expected end date derived from the position's duration.
→ *The applicant identity closes; the student identity opens on the same person_id.*

What can refuse the action, and why

- **Issue with visa_required, check incomplete** → 422 "complete the visa check before issuing".
- **Accept with pending conditions** → 422 with the count and first condition named.
- **Opportunity full** → 409 with the current places count.

The screenshot displays the PGR Platform interface for the 'Admissions' section. The left-hand sidebar contains a 'WORKSPACE' menu with items like Dashboard, Analytics, My journey, Tasks, Persons, Research, Recruitment, and Admissions (which is selected). Below this is an 'ADMINISTRATION' menu with Funding integrity, Statutory, and Workflows. The main content area is titled 'Admissions' and includes a sub-header 'Offers, acceptance, and onboarding.' A card titled 'Applications awaiting an admissions decision' contains a table with columns 'Application' and 'Stage'. The table is empty, and a message below it says 'Nothing awaiting a decision. Advance an application in Recruitment first.' The top of the page features a navigation bar with the PGR Platform logo, a search bar, and a notification bell. The bottom right corner has a blue 'Ask PGR' button.

Admissions — Accept creates the student record atomically.

2.5 · Students — record and lifecycle events (suspension / extension)

Purpose. Every registered PGR in one filterable list. The student record is the richest screen in the platform, carrying the whole journey.

Value it adds. One page: record, lifecycle changes, Pattern Lab prediction (sits beside deterministic flags), supervisors, meetings, milestones + panels + appeals, funding + payments + waivers, F4 classification, documents, history. Row scoping is enforced — supervisor sees only their supervisees; out-of-scope reads return 404 (not 403).

Before you start

- `student.read` to browse; `student.write` for edits; `student.lifecycle.approve` to approve suspensions.

Step-by-step

1. Open **Students**. Filter by department, cohort year, or search by name/student_ref.
2. Click a student row.
→ *The full record loads with the sections named above.*
3. **Request a suspension or extension**
4. On the record, in the **Lifecycle changes** card, click **+ Request...**
→ *Dialog: type (**suspension** / **extension** / **mode change**), from-date, to-date (for suspension) or new-end-date (for extension), reason.*
5. Submit. The event is created in state proposed; no timeline change yet.
→ *A task lands for a second admin with `student.lifecycle.approve`.*
6. A different admin opens the task, reviews the event, clicks **Approve**.
→ *The timeline recalculates from the original baseline (`original_expected_end_date` is immutable). Only **undecided** milestones shift; decided ones are historical facts and stay.*
7. The scheduler auto-returns expired suspensions on its 60-second cycle.
→ *The student's status flips back from `on_leave` to `active`; funding chase resumes.*

What can refuse the action, and why

- **Overlapping suspensions** → refused with the conflicting range named.
- **Same-user approval** (proposer = approver) → refused; a different user must approve.

PGR Platform
RESEARCH LIFECYCLE

FUSION PRACTICES ORACLE Partner **PGR Platform**

Q Search or jump to... Ctrl K

WORKSPACE

- Dashboard
- Analytics
- My journey
- Tasks
- Persons
- Research
- Recruitment
- Admissions
- Students**
- Supervision
- Progression
- Funding
- Thesis
- Completion

ADMINISTRATION

- Funding integrity
- Statutory
- Workflows

admin
admin@example.com

> Students

Omar Okafor
PGR student record.

← Back to students

Record

STUDENT REF SCALE-99-0300	STATUS active	STUDY MODE full time
START DATE 2025-09-04	EXPECTED END 2029-03-05	RESEARCH TOPIC Health Informatics: modelling study

Lifecycle changes + Request...

Suspensions, extensions and mode changes — and what they did to the timeline.

active Expected end 2029-03-05 — unchanged from the date agreed at registration.

No suspensions, extensions or mode changes have been requested for this student.

Predicted risk (Pattern Lab)
Advisory model output — it sits beside the deterministic indicators, never replaces them.

Funding Continuity Risk model scored 8/22/2026, 5:04:24 PM

experienced a funding gap

6%

Current supervisors -2.7 pp Stipend amount -1.4 pp

Advisory prediction beside the deterministic indicators — association, not causation; a human decides.

Ask PGR

Student record — the richest page in the platform.

2.6 · Supervision — assign, end, and read the caseload

Purpose. History-preserving supervisor assignment. Capacity guard prevents assigning beyond the institutional ceiling (default 8). Overdue flag is driven by the expected meeting interval (default 90 days) — both are editable in Settings.

Value it adds. History is preserved, never overwritten. Assign → End → Re-assign, all audited. The supervisor's own caseload is a live view of what needs their attention.

Before you start

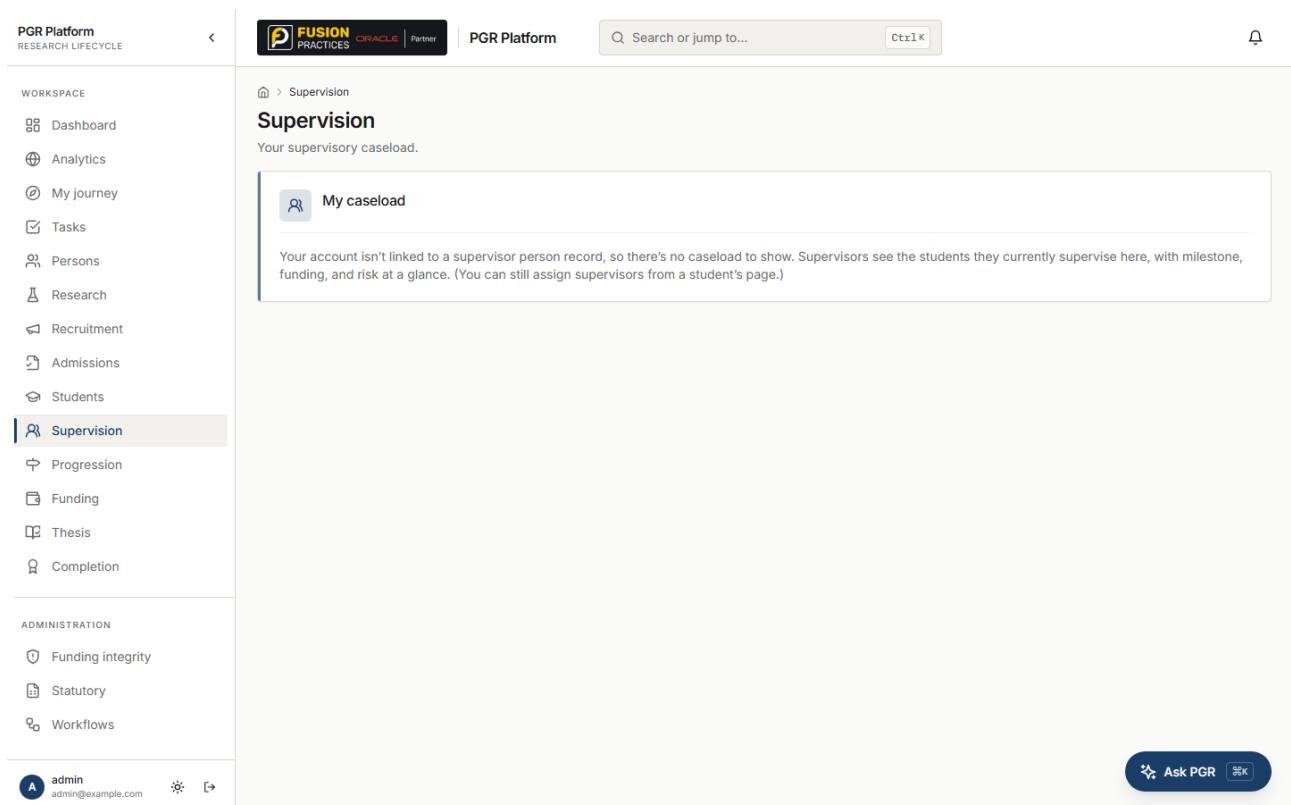
- student.write to assign; supervisor role to see caseload.

Step-by-step

1. Open the student record. Scroll to the **Supervisors** card.
2. In the **Choose a supervisor...** picker, pick a person with the Supervisor role.
→ Only current supervisors show; capacity is checked live — a supervisor at capacity is shown but with a red badge.
3. Pick the role (primary / co-supervisor). Click **Assign**.
→ The assignment is created; the previous supervisor of that role is not touched — call End on their row first if you want to replace, otherwise both stay open.
4. To end an assignment, click **End** in the supervisor row.
→ The relationship's valid_to is set to today; nothing is deleted. History reads correctly forever.
5. **Supervision caseload (supervisor view)**
6. Sign in as the supervisor. Open **Supervision** from the sidebar.
→ Only their own supervisees show. Columns: student, last meeting date, days since, overdue flag.
7. A row highlighted red is overdue against the institutional expected interval.
→ Suspended students are excluded from overdue chase (paused students are not chased).

What can refuse the action, and why

- **Capacity exceeded** → refused with the current caseload count for that supervisor.
- **Re-assigning the same person to the same role twice** → refused (idempotency).



Supervision caseload — row scoping means supervisor sees only their own supervisees.

2.7 · Progression — milestones, panels, and appeals

Purpose. Configurable milestones per programme (upgrade, annual review, viva). Panels require a chair + one independent assessor who is not a supervisor. Outcomes carry conditions with auto re-review. Appeals reopen a decided milestone.

Value it adds. Panel independence is enforced by the platform, not a checklist. Only undecided milestones shift when a suspension recalculates a timeline. Every decision is auditable.

Before you start

- `progression.read` to browse; `progression.decide` to record outcomes.

Step-by-step

1. Milestones are auto-generated by the worker on the 60-second cycle; you do not create them by hand.
2. **Convene a panel**
3. Open the student record. Scroll to the **Progression milestones** card. Find the due milestone.
→ *Click the row to expand.*
4. Click **+ Add panel member**. Pick a person; the platform refuses anyone who is one of the student's supervisors.
→ *Add roles chair and independent_assessor. The mandatory-composition rule is enforced.*
5. **Record the outcome**
6. Click **Decide**.
→ *Fill rationale, mandatory conditions (multi-line), and outcome letter.*
7. Pick outcome: **pass / conditions / fail**.
→ *Save. The outcome is recorded on the audit trail; conditions create an auto re-review milestone; the 28-day appeal window opens.*

8. Handle an appeal

- If the student appeals, from the student record click **+ Appeal** on the milestone.
→ *The milestone reopens; the outcome becomes appealed; a re-decide task lands.*

What can refuse the action, and why

- Assessor is also a supervisor** → refused with "independent assessor cannot be a current supervisor of this student".
- Missing chair or assessor** → decide refused with the missing role named.

The screenshot displays the PGR Platform interface. The top navigation bar includes the PGR Platform logo, a Fusion Practices Oracle Partner logo, a search bar with the text 'Search or jump to...', and a notification bell icon. The left sidebar is divided into 'WORKSPACE' and 'ADMINISTRATION' sections. The 'WORKSPACE' section includes links for Dashboard, Analytics, My journey, Tasks, Persons, Research, Recruitment, Admissions, Students, Supervision, Progression (highlighted), Funding, Thesis, and Completion. The 'ADMINISTRATION' section includes links for Funding integrity, Statutory, and Workflows. The main content area is titled 'Progression' and shows a list of milestones for 'PhD Computer Science': Induction Review (30 days after start), Confirmation Review (270 days after start), and Annual Progress Review (540 days after start). The bottom right corner features an 'Ask PGR' button.

Progression queue — the independent-assessor rule is a hard gate.

2.8 · Funding — arrangements, stipend approvals, fee waivers

Purpose. Funding over time, not a single field. A student can hold several arrangements in sequence or in parallel. Each has a payment schedule flowing due → approve → paid; approving emits a domain event to Finance via the outbox in the same transaction.

Value it adds. At-least-once event delivery to Finance. Fee waivers are audited. Cost centres and funder references are captured per arrangement.

Before you start

- funding.change to approve payments.

Step-by-step

- Open **Funding**. Filter by source or student.
- Add an arrangement** (from the student's funding card)
- Click **Add arrangement**. Fill funder, source, stipend amount, currency, cost centre, dates.
→ *The arrangement is created active; the payment schedule is generated according to the monthly/quarterly frequency you set.*
- Approve a due stipend**
- Under the arrangement, find the row in state **due**. Click **Approve**.

→ In one transaction: row → **approved**; outbox event funding.stipend.approved is written; the dispatcher will POST to Finance on its 20-second cycle.

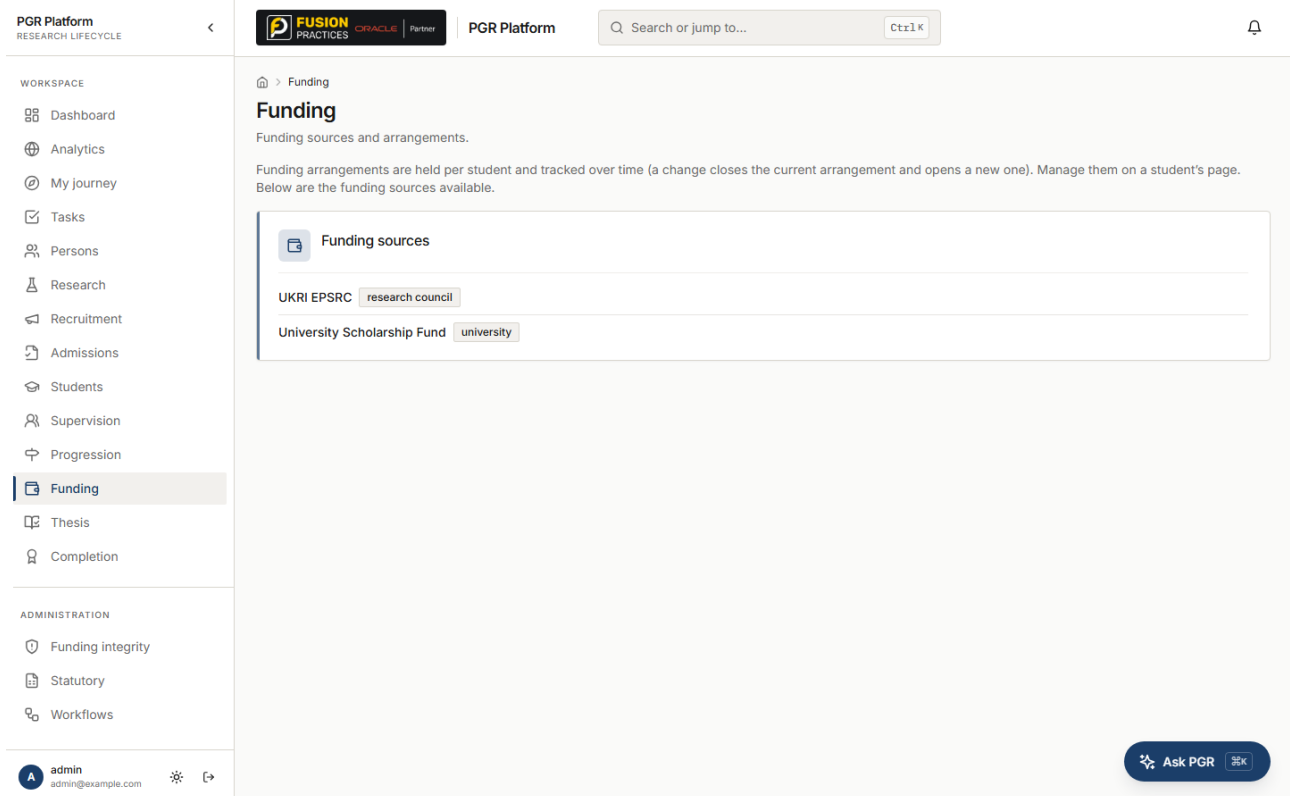
6. Once Finance settles (or you record settlement), the row moves to **paid**.

→ The audit trail records the approver + timestamp.

7. **Grant a fee waiver**

8. In the student's funding card, click **Waive fee**. Pick kind (full / percentage / amount), value, note.

→ The waiver is created; the payment schedule updates accordingly; the action is audited.



Funding arrangements. Approving a stipend emits to Finance in the same transaction.

2.9 · Funding integrity — the cohort health check

Purpose. Every student whose funding chain has a problem, errors first. Eight rules per student (gap between arrangements, over-payment, missing project link, expiring within threshold, and more) each carrying the values that produced them.

Value it adds. 0.17 s across 266 students (14× faster than the naïve per-student version). After a bulk import of arrangements, target is zero errors.

Before you start

- reporting.read.

Step-by-step

1. Open **Funding integrity** in the Administration sidebar.
2. Read the summary at the top: students checked · errors · warnings.
→ Rows are sorted errors first, then warnings, then clean.
3. Filter by error class (gap, over-payment, missing project link, expiring soon).
→ Each row shows the student, the rule that fired, and the values that triggered it ("gap of 12 days from 2025-04-01 to 2025-04-13").
4. Click a row to jump into the student's funding card.
→ Fix — usually by extending an arrangement, correcting cost centre, linking a project, or granting a waiver.

5. Re-run by refreshing this page; the checks run afresh from a bulk read.

PGR Platform RESEARCH LIFECYCLE

FUSION PRACTICES ORACLE Partner **PGR Platform** Search or jump to... Ctrl+K

WORKSPACE

- Dashboard
- Analytics
- My journey
- Tasks
- Persons
- Research
- Recruitment
- Admissions
- Students
- Supervision
- Progression
- Funding
- Thesis
- Completion

ADMINISTRATION

- Funding integrity**
- Statutory
- Workflows

admin admin@example.com

Funding integrity

Every active student's funding chain, checked end to end: project → award → funder → arrangement → stipend.

STUDENTS CHECKED **266** WITH FINDINGS **109** ERRORS **97** WARNINGS **25**

SEVERITY **All** Errors only

Students with funding findings
Informational notes are excluded here — this list is only what somebody has to act on.

Amara Abara SCALE-99-0145 active error 2025-05-03 → 2028-11-01
ERRORS (1)

124 days with no funding between 2026-06-07 and 2026-10-09.
from 2026-06-07 - to 2026-10-09 - days 124
funding_gap

Amara Costa SCALE-99-0262 active error 2026-01-04 → 2030-01-04
ERRORS (1)

77 days with no funding between 2027-02-08 and 2027-04-26.
from 2027-02-08 - to 2027-04-26 - days 77
funding_gap

Amara Zhao SCALE-99-0139 active error 2025-12-25 → 2029-12-25
ERRORS (1)

Funding runs to 2029-12-25 but award SCALE-AWD-99-018 ends 2029-03-06.
award SCALE-AWD-99-018
funding_outlives_award

Ask PGR

Funding integrity — every warning names the exact values that produced it.

2.10 · Thesis — examination pipeline with Col enforcement

Purpose. intention → submit → nominate examiners → viva → outcome → corrections. Examiners carry an affiliation; **conflict-of-interest** blocks approving a nominee who co-authored with the candidate or the supervisor. Corrections open at outcome time (minor 28 d / major 182 d) with sign-off.

Value it adds. Col is a hard block, not a warning. Corrections carry the correct clock automatically.

Before you start

- student .write to move the pipeline.

Step-by-step

1. Open **Thesis**. Filter by stage (submitted, under exam, awaiting corrections, complete).
2. **Declare intention** → **submit**
3. From the student record's Thesis card, click **Declare intention**. Enter the working title.
→ State becomes *intention_declared*.
4. When the thesis is ready, click **Submit**. Upload the PDF via Documents.
→ State becomes *submitted*.
5. **Nominate examiners**
6. Click + **Nominate examiner**. Pick person, affiliation (internal / external), role (chair / examiner).
→ The platform runs the Col check: is this person a supervisor of the candidate? Have they co-authored? If yes, nomination is refused with the reason.
7. Once at least two examiners are nominated and clean, click **Approve nominations**.
→ State becomes *examiners_approved*.
8. **Viva and outcome**
9. Click **Schedule viva** — datetime, format (in person / video), location.
→ State becomes *viva_scheduled*.

10. After the viva, click **Record outcome**. Pick **pass** / **pass_with_corrections** / **major_corrections** / **fail**.
→ *Corrections outcomes open a thesis_correction row with the correct deadline (28 d / 182 d) and a task for the candidate. Pass moves the pipeline forward to graduation.*
11. **Sign off corrections**
12. When the candidate submits corrections (through the portal), the row moves to **submitted**.
→ *The supervisor reviews and clicks **Approve**; the row moves to **approved** and thesis moves to **complete**.*

What can refuse the action, and why

- **Col on a nominated examiner** → refused with the reason ("co-authored with candidate", "supervisor of candidate").
- **Corrections deadline missed** → task escalates on the worker's next cycle.

The screenshot shows the PGR Platform interface. The sidebar on the left contains a 'WORKSPACE' section with links to Dashboard, Analytics, My journey, Tasks, Persons, Research, Recruitment, Admissions, Students, Supervision, Progression, Funding, Thesis, and Completion. Below this is an 'ADMINISTRATION' section with links to Funding integrity, Statutory, and Workflows. The main content area is titled 'Thesis & examination' and includes a sub-header 'Students'. Below this, there is a table with columns for 'Student' and 'Status'. The table lists several student IDs with their corresponding statuses: SCALE-99-0300 (active), SCALE-99-0299 (withdrawn), SCALE-99-0298 (withdrawn), SCALE-99-0297 (withdrawn), SCALE-99-0296 (active), SCALE-99-0295 (active), SCALE-99-0294 (active), SCALE-99-0293 (active), SCALE-99-0292 (withdrawn), SCALE-99-0291 (registered), and SCALE-99-0290 (active). Each row has an 'Open' button to its right. At the bottom right of the table, there is a button labeled 'Ask PGR'.

Theses list. Col enforcement is a hard block.

2.11 · Completion — classification workflow + certificate PDF (F4)

Purpose. A three-step state machine (propose → confirm → publish) with approver separation. Publishing renders the certificate PDF into the object store and attaches it to the award. Graduation is refused until classification is **published**.

Value it adds. Records Office stops running a parallel Word template. Certificate download is a real PDF, not a placeholder.

Before you start

- Chair holds `student.write`; exam board holds `progression.decide`; Registry holds `reports.signoff`. Approver separation is enforced.

Step-by-step

1. Open the student record. Scroll to **Award classification (F4)**.
2. **Chair proposes**

3. Chair picks a classification from the dropdown (PhD, PhD-with-corrections, PhD-with-major-corrections, MPhil, MRes).
→ Only the five whitelisted values render; an unknown value is refused on the API side.
4. Click **Propose**.
→ The Award row is created if none exists; state moves from draft to proposed; the chair's user id is recorded as proposer.
5. **Exam board confirms**
6. A different user (exam board, holds progression.decide) opens the same student, sees state *Proposed* — awaiting exam board.
→ The Confirm button renders for them; approver separation is enforced server-side.
7. They click **Confirm (exam board)**.
→ State moves to confirmed. If it is the same user as the proposer, refused: "the confirmer must differ from the proposer".
8. **Registry publishes**
9. Registry (holds reports.signoff) opens the same student, sees state *Confirmed* — awaiting Registry publish.
→ The Publish button renders.
10. They click **Publish (Registry)**.
→ The certificate PDF is rendered by reportlab, saved into the object store, linked to the award via certificate_document_id. State moves to published with the publication timestamp.
11. The **Download certificate** link appears; anyone with student.read can download the real PDF.
→ The PDF is a proper 3-page A4 document with the person's name, the classification and the award date.

What can refuse the action, and why

- **Same-user confirm** → 422 "approver separation: the confirmer must differ from the proposer".
- **Publish without confirm** → 422 "only a confirmed classification can be published".
- **Unknown classification** → 422 with the allowed list.
- **Graduation before publish** → 422 "award classification must be published before graduation".

F4 — Award classification card between Thesis & completion and Relationship map.

2.12 · Completion — the atomic graduation transaction

Purpose. One transaction: award recorded, funding closed, student identity closed, alumnus identity opened. All under the same `person_id`.

Value it adds. Ceremonies and final awards live here. The action is irreversible without database surgery — check the classification is published before pulling the trigger.

Before you start

- `student.write`; classification must be published (F4).

Step-by-step

1. Open **Completion**. Filter by ceremony or cohort.
2. Find the completion row for the student. Confirm the classification is **published** in the F4 card.
→ If not, complete the F4 workflow first — otherwise Graduate refuses.
3. Click **Graduate**.
→ The transaction fires: `student.status = completed`; *student identity closed*; *alumnus identity opened*; *funding arrangements end*; *outbox event student.graduated is written for Finance and Records Office*.
4. Reload the person in **Persons**.
→ The *alumnus identity is now current*; the *student identity has a valid_to*.

PGR Platform
RESEARCH LIFECYCLE

PGR Platform

WORKSPACE

Dashboard

Analytics

My journey

Tasks

Persons

Research

Recruitment

Admissions

Students

Supervision

Progression

Funding

Thesis

Completion

ADMINISTRATION

Funding integrity

Statutory

Workflows

admin

admin@example.com

Completion

Completion & graduation

Confirm completion and graduate.

Students

Confirm completion and graduate on the student's page. Graduation records the award, closes funding, marks the student completed, and opens the alumni identity.

Student	Status	
SCALE-99-0300	active	Open
SCALE-99-0299	withdrawn	Open
SCALE-99-0298	withdrawn	Open
SCALE-99-0297	withdrawn	Open
SCALE-99-0296	active	Open
SCALE-99-0295	active	Open
SCALE-99-0294	active	Open
SCALE-99-0293	active	Open
SCALE-99-0292	withdrawn	Open
SCALE-99-0291	registered	Open
SCALE-99-0290	active	Open

Ask PGR

ASK

Completions queue. Graduation is atomic.

2.13 · Tasks — SLA snapshot and per-row badge (F5)

Purpose. Your personal work queue plus F5's SLA clock. Tasks can carry a target turnaround; a worker sweep marks breached; the report shows within-target rate.

Value it adds. Institutional service levels (turnaround on offer decisions, corrections sign-off, viva scheduling) are reported for the first time. SLA meetings stop being a manual reconstruction from audit logs.

Before you start

- reporting.read to see the report; admin.configure to attach an SLA to a task.

Step-by-step

- Open **Tasks**.
- Read the **SLA snapshot (F5)** card: tasks with SLA · open with SLA · breached · within-target rate.
→ The rate is a live snapshot (60-second refresh).
- In the task list, each row with an SLA shows a Timer badge.
→ Green Timer + hours = within target; red "SLA breached" = past target.
- Attach an SLA to a task** (Admin only)
- Call POST /api/v1/tasks/{id}/sla with target_seconds and working_days_only.
→ The task's clock starts (or is picked up from the task's created_at). The sweep marks it breached if already over.
- Sweep breached tasks**
- The worker runs the sweep on its cycle; you can force a run with POST /api/v1/tasks/sla-sweep.
→ Idempotent — running twice does nothing new. Returns a count of newly breached tasks.
- Complete a task**
- Click **Complete** on the row.
→ State moves to done; the SLA report updates on next refresh.

PGR Platform — Role-based User Manual (detailed)

Page 30

WORKSPACE

- Dashboard
- Analytics
- My journey
- Tasks**
- Persons
- Research
- Recruitment
- Admissions
- Students
- Supervision
- Progression
- Funding
- Thesis
- Completion

ADMINISTRATION

- Funding integrity
- Statutory
- Workflows

Tasks & notifications

Work assigned to you and your roles.

SLA snapshot (F5)
Turnaround against institutional service levels — the platform's own promise back to the department.

TASKS WITH SLA	OPEN WITH SLA	BREACHED	WITHIN-TARGET RATE
5	5	1	80%

My task queue

Plan funding renewal — arrangement expires 2026-10-19	PGR Administrator	Complete
Plan funding renewal — arrangement expires 2026-10-29	PGR Administrator	Complete
Plan funding renewal — arrangement expires 2026-10-22	PGR Administrator	Complete
Plan funding renewal — arrangement expires 2026-09-28	PGR Administrator	Complete
Plan funding renewal — arrangement expires 2026-08-25	PGR Administrator	Complete
Plan funding renewal — arrangement expires 2026-09-14	PGR Administrator	Complete
Plan funding renewal — arrangement expires 2026-10-14	PGR Administrator	Complete
Plan funding renewal — arrangement expires 2026-09-29	PGR Administrator	Complete
Plan funding renewal — arrangement expires 2026-09-02	PGR Administrator	Complete

admin
admin@example.com

Ask PGR

F5 — SLA snapshot: 5 tasks with SLA, 1 breached, 80% within target.

2.14 · Integration hub — dispatcher, dead letters, bulk replay (F5)

Purpose. Every outbound event (funding → Finance, updates → Research) and every inbound signed webhook (HMAC-SHA256) is visible here. F5 adds **bulk replay**: select many dead-letters, replay in one audited call.

Value it adds. Instead of clicking Replay N times when a partner endpoint recovers, tick the boxes and click once. Every replay is audited with actor and outcome.

Before you start

- admin.configure.

Step-by-step

1. Open **Integration** from the Administration sidebar.
2. Read the dispatcher status at the top: pending events · dispatched last 15 min · dead-lettered count.
→ Any pending > 60 seconds is worth PGR investigating in the worker log.
3. **Bulk replay**
4. Scroll to **Dead letters**. Each row shows event type, attempts, last error, created-at, and a checkbox.
→ The *Replay* button beside each row is the single-item replay; bulk replay uses the checkboxes.
5. Tick the checkboxes on the rows you want to replay, or click **Select all** in the header.
→ The counter next to *Select all updates*.
6. Click **Replay N selected**.
→ The bulk endpoint runs each replay; the result names how many succeeded per event id. The toast reports "Replayed N / M".
7. Refresh — successfully replayed events disappear from the dead-letter list; the dispatcher will retry them on the next 20-second cycle.
→ If a specific event failed to replay (e.g. already recovered on its own), the outcome row explains.
8. **Investigate a partner failure**
9. Under the row, expand **Last error** to see the partner's response.
→ Malformed payloads land as failed with the parse error; never dropped silently.

Integration hub
Outbox dispatcher, adapters, and inbound webhooks.

Integration reconciliation healthy
What needs attention at the integration boundary — last 30 day(s).

Integration boundary is healthy — nothing pending or failed.

PENDING	DISPATCHED (IN WINDOW)	DEAD-LETTERED
0 nothing waiting to go out	5 last 30 day(s)	0 exhausted retries — replay below

DEAD LETTERS
No dead-lettered events. Every outbound event has been delivered or is still retrying.

TRAFFIC BY SYSTEM

System	Direction	Status	Count
finance	inbound	success	1
finance	outbound	success	2
hr	inbound	success	1
internal	outbound	skipped	3
research	inbound	success	1

FAILED INBOUND MESSAGES
No inbound message failed in the last 30 day(s).

Ask PGR

Integration hub. Bulk replay via Select-all + Replay N selected.

2.15 · Statutory — day-to-day (map, validate, generate, clone)

Purpose. You edit the profile, run validate, and produce the CSV. Registry / HESA does the sign-off (F1). Cloning to a new year preserves the prior year byte-identical.

Value it adds. Statutory changes stop being code releases; regenerating a prior year uses that year's mapping and reproduces the original file.

Before you start

- `admin.configure` to create/edit; `reporting.read` to validate/generate.

Step-by-step

1. Open **Statutory**. Pick a profile (or clone last year's).
2. **Add / edit field mapping**
3. Click **+ Add field**. Fill target field (e.g. STUID), source expression (e.g. `student.ref`), transform, default, required, allowed values.
 - Source expression is a dotted path over the flat student record — not an expression language. Unknown transform is refused at add time.
4. To edit an existing mapping, click the pencil icon on the row.
 - Changes are refused if the profile is signed off — *unsign first*.
5. **Validate**
6. Click **Validate**.
 - The platform runs every student against every field and shows a report: rows checked · errors per student per field. Fix the errors listed.
7. **Generate**
8. Click **Generate**.
 - The CSV is produced as an export job; download from **Integration** → **Exports**.
9. **Clone to a new year**

10. Click **Clone to new year** in the header. Enter the new academic year (e.g. 2027/28).

→ The clone copies every mapping; the source year stays exactly as it was.

Statutory returns

A statutory return is configuration, not code — HESA is an external specification, expressed as a versioned profile of field mappings.

Report profiles
Versioned by academic year. Next year's change is an edit here, not a code release.

Code	Name	Academic year	Version	Fields	Status
HESA_STUDENT	HESA Student Return	2026/27	v1	12	active

HESA_STUDENT — 2026/27
Target field ← source expression + transform + validation.

#	Target field	Source expression	Transform	Required	Allowed values	Default
1	HUSID	student.ref	—	required	—	—
2	SURNAME	person.familyName	upper	required	—	—
3	FNAME	person.givenName	upper	required	—	—
4	NATION	person.nationality	upper	—	—	—
5	COURSEID	programme.code	—	required	—	—
6	INSTUFE	funding.type	lower	—	—	—
7	STULOAD	student.mode	lower	required	full_time, part_time	—
8	COMDATE	student.startDate	date_compact	required	—	—
9	ENDDATE	student.expectedEndDate	date_compact	—	—	—

Ask PGR

Statutory profile list. Cloning to a new year preserves the prior year exactly.

2.16 · Audit — filtering the immutable trail

Purpose. Every privileged action. Actor from the JWT (no DB hit). Filter by user, action, entity, or request id.

Value it adds. Complete forensic reconstruction of any incident. Trails are never deleted — even after a person leaves.

Before you start

- `audit.read`.

Step-by-step

1. Open **Audit**.
2. Filter by user (email), action prefix, entity type, or request id (paste from server log).
→ The table refreshes on each filter change; filters combine (AND).
3. Click a row to expand and see the request payload snippet and the response status.
→ Trails are ordered newest first by default.

WORKSPACE

-  Dashboard
-  Analytics
-  My journey
-  Tasks
-  Persons
-  Research
-  Recruitment
-  Admissions
-  Students
-  Supervision
-  Progression
-  Funding
-  Thesis
-  Completion

ADMINISTRATION

-  Funding integrity
-  Statutory
-  Workflows

 admin
admin@example.com > Audit

Audit

Immutable record of privileged actions and state changes.



Filters

Entity type

Entity ID

Actor email

Time	Actor	Method	Action	Entity	Status
2026-08-22 11:58:14	admin@example.com	POST	POST /api/v1/pattern-lab/models/c882eadc-c184-4317-8eda-4ca3d54dc2b3/retrain	pattern-lab - c882eadc	200
2026-08-22 11:55:06	admin@example.com	POST	POST /api/v1/pattern-lab/models/c882eadc-c184-4317-8eda-4ca3d54dc2b3/retrain	pattern-lab - c882eadc	200
2026-08-22 11:14:18	admin@example.com	POST	POST /api/v1/pattern-lab/versions/9b18c948-d2d2-42b9-81f0-24dbc80e28fc/transition	pattern-lab - 9b18c948	200
2026-08-22 11:13:40	admin@example.com	POST	POST /api/v1/pattern-lab/versions/9b18c948-d2d2-42b9-81f0-24dbc80e28fc/transition	pattern-lab - 9b18c948	200
2026-08-22 11:13:06	admin@example.com	POST	POST /api/v1/pattern-lab/versions/9b18c948-d2d2-42b9-81f0-24dbc80e28fc/transition	pattern-lab - 9b18c948	200
2026-08-22 10:44:35	admin@example.com	POST	POST /api/v1/pattern-lab/datasets/a1b254b2-4afc-4765-83bd-d5abdfc35aa7/discover	pattern-lab - a1b254b2	200
2026-08-22 10:44:27	admin@example.com	POST	POST /api/v1/pattern-lab/datasets	pattern-lab	201
2026-08-22 10:23:39	admin@example.com	POST	POST /api/v1/pattern-lab/datasets/fcbe4914-ecb9-4748-926d-f28df09f118a/discover	pattern-lab - fcbe4914	200
2026-08-22 10:23:26	admin@example.com	POST	POST /api/v1/pattern-lab/datasets	pattern-lab	200
2026-08-22 09:03:13	admin@example.com	POST	POST /api/v1/research/supervisor-suggestions	research	200

 Ask PGR

25K

Audit trail with filter row on top.

Supervisor

Row-scoped role — sees only their own supervisees. Owns the working relationship with each student: meetings, progression decisions, exam-board classification confirmation.

3.1 · Supervision caseload — read your queue

Purpose. Every supervisee in one view, with last meeting and overdue flags.

Value it adds. Overdue is a fact, not a judgement — the platform reads the institutional expected interval and reports it consistently.

Before you start

- Supervisor role. Row scoping restricts to your own supervisees.

Step-by-step

1. Open **Supervision** from the sidebar.
2. Read the columns: **Student**, **Last meeting**, **Days since**, **Overdue**.
→ Rows in red are overdue against the default 90-day expected interval (editable in Settings).
3. Click into any supervisee to jump to their record.
→ You land on their student detail with full context (row-scoped — you cannot see other students).
4. The caseload live-refreshes on 30-second interval.
→ Meetings logged elsewhere flip the overdue flag off automatically on the next refresh.

The screenshot shows the PGR Platform interface. The top navigation bar includes the PGR Platform logo, a search bar, and a notification bell. The left sidebar lists various workspace and administration options. The 'Supervision' option is highlighted in the sidebar. The main content area shows the 'Supervision' page with a message indicating that no caseload is visible because the user's account is not linked to a supervisor person record.

Caseload — row scoping keeps this list focused.

3.2 · Log a supervision meeting

Purpose. First-class meeting log on every student. Format, duration, notes, actions agreed.

Value it adds. 90-day compliance becomes real. The supervisor-student conversation stops living in Outlook.

Before you start

- Supervisor of this student (row scoping).

Step-by-step

1. Open the student record. Scroll to **Supervision meetings**.
2. In the **Record a meeting** row, fill: date (defaults to today), supervisor (defaults to you), format (in person / video / phone), duration in minutes.
→ *Date cannot be in the future; duration must be > 0.*
3. In **Discussion notes**, type the substantive content.
→ *Free text; searchable via audit.*
4. In **Actions agreed**, type the follow-ups (bullet list is fine).
→ *These land in the student's task queue on save.*
5. Click **Record meeting**.
→ *The row appears at the top of the meeting log immediately; the overdue flag on your caseload lifts.*

What can refuse the action, and why

- **Future date** → refused.
- **Zero duration** → refused.
- **Row-scope miss** (not their supervisor) → 404 on the student.

The screenshot shows the PGR Platform interface for a student record. The sidebar on the left contains navigation links for Workspace (Dashboard, Analytics, My journey, Tasks, Persons, Research, Recruitment, Admissions, Students) and Administration (Funding integrity, Statutory, Workflows). The main content area is titled 'Students' and shows the record for 'Omar Okafor'. The record includes fields for Student Ref (SCALE-99-0300), Status (active), Study Mode (full time), Start Date (2025-09-04), Expected End (2029-03-05), and Research Topic (Health Informatics: modelling study). Below the record, there are sections for 'Lifecycle changes' and 'Predicted risk (Pattern Lab)'. The 'Predicted risk' section shows a 'Funding Continuity Risk model' with a 6% risk level and a bar chart. The interface is branded with Fusion Practices and Oracle logos.

Supervision meetings card — same UI whether admin or supervisor.

3.3 · Progression — decide milestones and F4 confirm

Purpose. Record milestone outcomes and — F4 — confirm an exam-board classification.

Value it adds. Approver separation is enforced by the platform: an exam board member cannot confirm a classification they proposed themselves.

Before you start

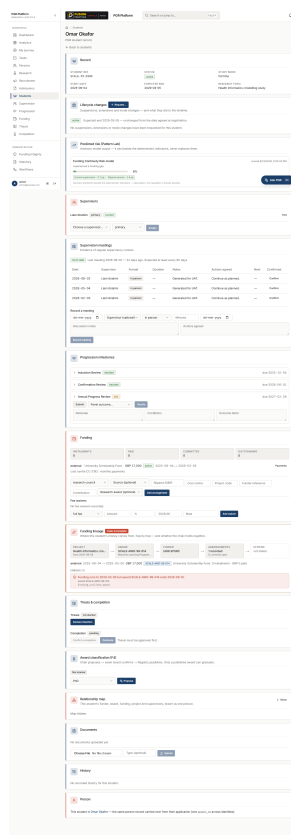
- progression.decide.

Step-by-step

1. **Decide a milestone**
2. Open the student. In **Progression milestones**, expand a due row.
→ *The Decide button and the fields (rationale, conditions, outcome letter) appear.*
3. Fill rationale (why the decision), conditions (mandatory follow-ups), outcome letter (student-facing).
→ *Click **Decide** and pick **pass** / **conditions** / **fail**.*
4. The outcome is recorded; conditions create an auto re-review milestone; the appeal window opens.
→ *You can no longer edit the decision — appeals must go through the appeal flow.*
5. **Confirm a classification (F4)**
6. Open the student. In **Award classification (F4)**, if the state is *Proposed — awaiting exam board*, the Confirm button is visible to you.
→ *If the proposer is you, the button is refused when clicked (approver separation).*
7. Click **Confirm (exam board)**.
→ *State moves to confirmed; the audit trail records you as the confirmer; Registry publish becomes possible next.*

What can refuse the action, and why

- **Self-confirm** → 422 approver separation.
- **Missing progression.decide** → the button never renders.



F4 — Confirm (exam board) with approver separation.

Student

The self-service role. Sees only themselves — milestones, meetings, funding, thesis, tasks. Row scoping means `student.read` is scoped to their own record.

4.1 · My journey — the student portal

Purpose. Every action the office takes about a student is visible to that student, in one page.

Value it adds. No more second-hand news. Every action is audited under the student's own `person_id`.

Before you start

- Student role.

Step-by-step

1. Sign in — you land on **My journey** by default.
2. Read your **Milestones** card top-down.
 - Due milestones show their date and the panel members already added; decided milestones show their outcome and any conditions.
3. Read your **Supervision meetings** — every past meeting your supervisor logged.
 - If a meeting is missing that you had, ask your supervisor to log it.
4. Read your **Funding schedule** — every stipend row with its status.
 - Approved rows will pay via Finance; you do not act here — the office does.
5. Read your **Thesis** card — current stage, examiners once approved, viva date, corrections status.
 - Corrections you owe show the deadline (28 d minor / 182 d major).
6. Read your **Tasks** — anything the platform is asking you to do.
 - Click a task for the action (upload document, acknowledge condition, complete a form).

The screenshot shows the PGR Platform interface for a student. The left sidebar lists various workspace and administration options. The main content area is titled 'My journey' and displays a message indicating that the user's account is not yet linked to a person record, preventing them from viewing their journey. The top of the page features the PGR Platform branding and a search bar. The bottom of the page shows the user's identity as 'admin' and a button to 'Ask PGR'.

Student portal — every action is audited under the student's own `person_id`.

4.2 · My preferences — quiet hours (F6)

Purpose. Under Settings → My preferences, control email delivery, per-event mute, daily digest, and — F6 — quiet hours. Non-urgent emails are suppressed inside the window.

Value it adds. Notification fatigue drops. Midnight-wrap windows work (e.g. 22:00 → 07:00). A hard email bounce elsewhere deactivates your email channel automatically; you see that state here.

Before you start

- Any authenticated user.

Step-by-step

1. Open **Settings** → **My preferences**.
2. **Basic controls**
3. Tick **Email notifications enabled** to receive email; leave unticked for in-app only.
→ *In-app is always on regardless.*
4. Tick **Daily digest** to batch non-urgent notifications into one morning email.
→ *Urgent notifications still go individually.*
5. **Quiet hours (F6)**
6. In the **Quiet hours (F6)** block, click the **From** time input. Pick a start (e.g. 22:00).
→ *The time picker uses your browser's native control.*
7. Click the **To** time input. Pick an end (e.g. 07:00).
→ *A window that wraps midnight (start > end) works correctly.*
8. Leave either blank to disable the window. Click **clear** to reset both.
→ *With no window, emails are delivered on schedule.*
9. **Mute specific events**
10. Tick the events you never want emailed (milestone decided, task assigned, etc.).
→ *In-app for those events still arrives.*
11. Click **Save preferences**.
→ *Preferences take effect on the next notification the platform sends you.*

Institution configuration, reference data, user administration and your personal preferences.

[List of values](#)
[Institution policy](#)
[Users & roles](#)
[My preferences](#)

☐ Daily digest (batch into one email)

QUIET HOURS (F6)

Non-urgent emails are suppressed inside this window. In-app notifications still arrive. Leave blank to disable.

From  To 

MUTE SPECIFIC EVENTS

☐ Milestone decided☐ Task escalated☐ Thesis outcome☐ Task assigned☐ Funding expiring☐ Supervisor assigned

Save preferences

Executive

Reporting-only role. Sees the whole estate at a glance; cannot mutate. Uses Enterprise 360 and the explainable risk model to brief the board.

5.1 · Executive dashboard (read-only)

Purpose. Same six lifecycle metrics + four administrator queues the admin sees, but write actions are hidden.

Value it adds. Board-ready snapshot in one page.

Before you start

- Executive role (reporting.read).

Step-by-step

1. Land on /dashboard after sign-in.
2. Read the six metric tiles.
 - Numbers are computed live from read replica; no polling here.
3. Scan the queues.
 - Click into a queue for a filtered read-only list of what needs attention.

The screenshot displays the PGR Platform Executive dashboard. The top navigation bar includes the PGR Platform logo, a search bar, and a notification bell. The left sidebar lists various workspace and administration options. The main content area is titled 'Executive dashboard' and provides a high-level overview of the PGR lifecycle. It features three main sections: Platform connectivity, Lifecycle metrics, and Administrator queues. The Lifecycle metrics section contains six tiles showing key performance indicators, and the Administrator queues section contains four tiles showing pending actions.

Lifecycle metrics					
ACTIVE RESEARCHERS 266 registered / active	APPLICATIONS 344 in pipeline	CONVERSION RATE 99.7% applicant → student	FUNDED STUDENTS 2 active arrangement	THESES SUBMITTED 68 awaiting/under exam	COMPLETIONS 31 graduated

Administrator queues			
AWAITING ASSESSMENT 1 applications	OFFERS TO ACCEPT 0 issued	REVIEWS DUE 0 progression	THESES SUBMITTED 68 to examine

Dashboard — Executive sees the same tiles read-only.

5.2 · Analytics — Enterprise 360 (five lenses)

Purpose. Volumes, Progression, Funding, Completion, Risk. Risk scoring is explainable — every at-risk student comes with the rule-based reasons.

Value it adds. Board reports stop being manually built in Excel. Risk is defensible.

Before you start

- `reporting.read`.

Step-by-step

1. Open **Analytics**.
2. Pick a lens from the tabs: Volumes → Progression → Funding → Completion → Risk.
→ *Each lens has its own charts and tables, all backed by the same aggregated queries.*
3. In the Risk lens, hover a tile to see the rules that fired for that risk band.
→ *Rules are the deterministic set (funding gap, meeting overdue, milestone slipped, thesis stalled).*
4. Click a student in the at-risk list to jump into their record.
→ *Executive sees the record read-only; write actions are hidden.*

The screenshot shows the PGR Platform interface. The top navigation bar includes the PGR Platform logo, a search bar, and a notification bell. The left sidebar contains a 'WORKSPACE' section with links to Dashboard, Analytics (selected), My journey, Tasks, Persons, Research, Recruitment, Admissions, Students, Supervision, Progression, Funding, Thesis, and Completion. Below this is an 'ADMINISTRATION' section with links to Funding integrity, Statutory, and Workflows. The main content area is titled 'Analytics' and 'PGR Enterprise 360, risk, and forecasting.' It features a 'Risk & completion' lens with a summary of key metrics: 264 Students at Risk (99.2% of active), 2 On Track (active, no risk flag), 31 Completions (9% rate), and 266 Active Researchers (registered / active). Below the summary is a list of 'AT-RISK STUDENTS' with their names, UAT IDs, and risk flags (e.g., 'no active funding', 'milestone overdue'). A search bar and a 'Ask PGR' button are also visible.

Enterprise 360 — read-replica routing keeps analytical reads off the write path.

5.3 · Pattern Lab (read-only for Executive)

Purpose. Advisory ML predictions surface beside deterministic risk indicators. Executive has `ml.read` but not `ml.approve`.

Value it adds. Board briefing on model health without touching decisions.

Before you start

- `ml.read`.

Step-by-step

1. Open **Advanced** → **Pattern Lab**.
2. Under **Monitoring**, read PSI drift and calibration for the production model.
→ *Health verdict names its reasons — a matured AUC drop or feature drift is called out.*
3. Predictions appear alongside deterministic risk flags on each student record.
→ *The advisory score never replaces the deterministic indicators — it sits beside them.*

PGR Platform

RESEARCH LIFECYCLE

- WORKSPACE
- Dashboard
- Analytics
- My journey
- Tasks
- Persons
- Research
- Recruitment
- Admissions
- Students
- Supervision
- Progression
- Funding
- Thesis
- Completion

ADMINISTRATION

- Funding integrity
- Statutory
- Workflows

PGR Platform

Search or jump to...

Pattern Lab

Governed pattern discovery over the institution's own lifecycle data — every number carries its evidence.

+ New analysis

- Overview
- Discover
- Models
- Predictions
- Monitoring

Active models 2 models

Best version per model — governed from candidate to production in the Models tab.

Model Name	Status	Algorithm	AUC
Progression Delay Risk model	candidate	Gradient boosting	0.609
Funding Continuity Risk model	candidate	Gradient boosting	0.702

Recent discoveries

Statistically significant patterns from the latest discovery runs.

+ New analysis

- Students with stipend amount at or below 18250 experienced a funding gap at 36%, vs 22% for above 18250 — 1.7× the rate.
significant | 1.7× the rate | Funding
- Students with stipend amount at or below 18250 experienced a funding gap at 36%, vs 22% for above 18250 — 1.7× the rate.
significant | 1.7× the rate | Funding

Data health

Whether each governed question has enough labelled history to analyse honestly.

Ask PGR

Appendix · Cross-role conventions

Row scoping

Supervisor sees only their supervisees; student sees only themselves; executive sees reporting only; PGR/Institution Admins see everything. Out-of-scope reads return **404**, not 403, so record existence isn't leaked. This applies uniformly on every list, detail and export endpoint.

Auth & permissions cheat sheet

Permission	What it grants
person.read / person.write	Persons, relationships, timelines
person.gdpr	F2 — merge, subject-access export, erase
student.read / student.write	Student record + everyday admin
student.lifecycle.approve	Approving suspensions/extensions (moves dates)
recruitment.read / recruitment.write	Opportunities, applications, F3 references + interviews + conditions
funding.read / funding.change	Funding arrangements + payments + waivers
progression.read / progression.write	Milestones and F4 classification confirm
reporting.read	Dashboards, Enterprise 360
reports.signoff	F1 statutory sign-off + F4 classification publish
admin.configure	Settings — institution policy, LOV, workflows
audit.read	Read the audit trail
assistant.use	Ask PGR + F6 write intents
ml.read / ml.analyse / ml.train	Pattern Lab (read, discovery, training)
ml.approve	Promote a Pattern Lab candidate to production

Consequence: A permission added to a role reaches a live session only after the token refreshes (≤ 15 min) or the user signs in again. To evict immediately, deactivate the user and run logout-all.

Running the platform locally

Batch file	What it does
start-all.bat	Postgres + backend + worker + frontend + open browser
start-backend.bat	FastAPI on http://127.0.0.1:8000
start-worker.bat	Background jobs (scheduler + outbox + notifications)
start-frontend.bat	Next.js on http://localhost:3000
stop.bat	Stop everything the batch files started
test.bat	Run backend test suite

Companion documents. PGR_Platform_Critical_Gaps.pdf — workarounds still to close.
PGR_Platform_MakeItReal_Plan.pdf — infra plan. PGR_Platform_Functional_MakeItReal_Plan.pdf — the fullstack plan F1–F6 delivered against.